

The Pillar Data Layer: A Multi-Model Streaming Database Turned Inside-Out for a Trustless Federation

pillar engineering (machine-checked design note)

2026-09-14

Abstract

Samza and Kleppmann’s “turning the database inside out” showed that an immutable log of events plus materialized views folded from it is a better factoring than a mutable-state database: the log is the source of truth, views are derived, and caches and indexes stop being things you invalidate by hand and become things you *rebuild*. Samza did this inside a trusted datacenter on a central broker (Kafka). Pillar does the same factoring for a **trustless, serverless, cryptographically-verifiable federation of PGP-trusted peers**—no broker, every record signed and content-addressed, confidentiality and authorization in the record format, convergence and coordination model-checked in TLA⁺. The user-visible consequence, and the point of this note, is that Pillar offers users the *same* data primitives it builds itself out of, over the *same* substrate (the `PillarMessage` envelope), with a real inspect/query surface—there is no separate “user database” bolted next to bespoke internal stores. We specify a small, distinct primitive suite (a **keyed store** with K/V and Document surfaces, **SQL views** as a derived-read layer over it, and a **time-series** store), show how SQL over the document store works as catalog-as-data with append-only field-delta writes and migration-free schema change, place the one hard consistency requirement (a CP subset) as a Samza co-partition-and-fold ordered by the coordination core rather than a global lock, and fix the data-placement boundary at the *collection* so every layer inherits one pinning decision over node tags. A time-sequence diagram walks the common and the distinctive streamdb operations—write and gossip merge, concurrent last-writer-wins convergence, a CP exclusive admission, a query from a folded view, a new view over existing data, and content-addressed rehydrate. This note is the design of record; the original-design core is TLA⁺-gated per Pillar’s specify-then-build rule.

1 The inside-out thesis, for a trustless federation

The inside-out model inverts the database: instead of a mutable table mutated in place, an append-only log of immutable events is the source of truth, and every readable view is a deterministic fold of that log. Samza realized this on Kafka inside a trusted datacenter. Pillar keeps the factoring but changes the environment to a

peer-to-peer federation with no central owner and no trusted operator, and adds what that environment demands: every op is content-addressed (a real CIDv1 multihash) and signed by a PGP-rooted identity; reads and writes gate through a Web-of-Trust RBAC decider; bodies are cell-encrypted or recipient-sealed; and the convergence, ordering, coordination, and confidentiality invariants are model-checked in TLA⁺ before any Rust. The doctrine that organizes the rest of this note: **build each data primitive once, dogfood it internally, and offer it to users with a real inspect/query surface**—so each internal plane (manifests, RBAC, sessions, quota, identity, observability) is a *consumer* of the same primitives a user application consumes.

2 One substrate, one envelope

Everything in the data layer rides one substrate. The event log is a content-addressed CvRDT op-log (`pillar-streamdb`): each op is immutable, content-addressed, signed, and merged by set-union; a deterministic fold produces the materialized view. There is exactly one record on the wire and on disk—the `PillarMessage` envelope (`pillar-wire`, companion paper)—of which streamdb ops, observability signals, and every libp2p control message are variants; the convergent cell-seal keeps the `Cid` stable under encryption. The *hot path* is pubsub/gossip of these envelopes; the *cold/bootstrap path* is Pillar’s own embedded IPFS node (`pillar-ipfs`), content-addressed and self-hosted, off the public DHT. A restarting or new node rehydrates from IPFS-pinned sealed segments plus its custody-held key—there is no central broker to replay from. Everything below is a *model over this one substrate*: the “different databases” a user sees are different fold/query/retention disciplines over the same signed log, not different storage systems.

3 The primitive suite

An application does not need one storage primitive per CRDT type, nor nine different stores; it needs a small number of distinct *access patterns*. Pillar offers three query interfaces over two storage models over the one substrate (Table 1).

SQL is a query layer, not a third stored type. It holds no storage; it folds the keyed store’s collections into materialized views and caches them. Counters (SUM/GROUP BY), graph traversals (recursive joins over an

Storage model	Access pattern / discipline
Keyed store (K/V + Document surfaces)	point lookup / structured record; per-field LWW-register by HLC; AP default, CP opt-in; snapshot + log-tail replay, folds to a condensable current state
Time-series (TSDB)	time-windowed range, retention/downsample, past-query required; append-only immutable blocks, <i>no</i> condensable current state
Query interface	Over
Keyed get/scan/watch SQL views	the keyed store (point + prefix + subscribe) the keyed store, as a <i>derived-read</i> layer owning no storage: project/filter/join/aggregate/traverse into cached materialized views
PSL	the TSDB (the compact-text/structured signal query surface)

Table 1: The suite a user picks: two storage models, three query interfaces, one substrate. Both storage models persist to the *same* IPFS content store—what differs is the data model on top.

edge collection), and rollups are *queries*, not primitives. There is no graph database, no standalone counter, and no “log store” distinct from the substrate—the streamdb *is* the log. **Cache is not a user primitive** either: it is the (query, root)-keyed materialized-view cache that recomputes only when the source root changes, invisible in the user’s model.

4 SQL over the document store

The relational surface presents mutable flat rows, but storage is the append-only signed log; the illusion is maintained by folding.

The catalog is data. DDL is not special—it writes documents to a `__catalog` system collection, event-sourced and gossip-replicated like everything else. `CREATE TABLE app.users(...)` writes a table document `{schema, collection: 'app.users'}`; `CREATE MATERIALIZED VIEW` writes a view document `{source: ['app.users'], query, since}`. A node learns every database/table/view by folding `__catalog`.

Data → table is structural. A row’s key *is* (collection, id): the collection component is the table id, so a node never needs a side lookup to know which table a document belongs to. The op-log is indexed by collection (and indexed columns) so folding a table is not a full scan.

A row update appends a delta, never mutating a flat document. An UPDATE compiles to a put op carrying *only* the changed fields with an HLC. The flat row is derived by folding the key’s ops under per-field LWW:

```
op1 t1: put users/alice {name:Alice,
                        email:ax, role:admin}
op2 t2: put users/alice {email:alice@y}
⇒      {name:Alice(t1), email:alice@y(t2),
        role:admin(t1)}
```

Because each field is its own LWW-register, a single-column update rewrites nothing else, and concurrent updates to *different* fields both survive; same-field conflicts resolve by highest HLC. DELETE appends a tombstone; the view drops the row, the log keeps the event, and compaction later collapses tombstoned keys out of the snapshot. The only literally-flat stored artifacts—the in-memory view and the periodic snapshot segment—are derived and rebuildable, never edited in place.

Data → view is derived, not stored. A node serving a view does not consult per-document “which view?” pointers. It reads the view document, folds the source collection(s), applies the predicate/projection, and caches on (view-id, root(source)). *Views subscribe to sources; sources are oblivious to views.*

A new view over existing data needs no migration. A schema change is a new view over the unchanged source: `CREATE MATERIALIZED VIEW app.users.v2 AS SELECT id, name AS full_name, email FROM app.users`. Nothing is moved. Because `app.users`’s ops are immutable and content-addressed, the node building `users.v2` folds those same ops from the beginning (snapshot + log-tail) under the new projection; old and new views coexist and consumers cut over gradually. Missing columns are computed/defaulted in the projection or supplied by a non-destructive per-field backfill that merges by LWW without rewriting existing fields.

5 Ordering: HLC is a wire-format requirement

`OpLog::order` sorts by content-address, not time. That is fine for a grow-only set, but per-field LWW and any latest-write-wins semantics are arbitrary without a logical clock in the op payload. Every keyed-store put/tombstone op therefore carries a Hybrid Logical Clock timestamp, and the fold resolves same-field conflicts by highest HLC with a deterministic tiebreak (author/cid). This is a user-visible part of the op schema, gated with the wire format.

6 Consistency: AP by default, a CP subset by co-partitioned fold

Pillar chooses its CAP point *per view*. The AP majority—sessions, offers, manifests, config, user records, trust edges—is a CRDT keyed store: writes always succeed and merge on heal (`ViewPolicy::Relaxed`). The handful of genuine hard invariants—exactly-once admission, hard quota ceilings, uniqueness—are *not* a relational engine, a global lock, or foreign keys. They are Samza’s co-partition-and-fold: the invariant’s key goes in a `ViewPolicy::Strict` partition and the reducer enforces the invariant over that partition’s total order. The one Pillar-specific twist is that Samza gets per-partition total order for free from Kafka’s single leader per partition, whereas Pillar’s substrate is *leaderless*; a Strict partition therefore takes its total order from the **coordination core**—a quorum-fenced lease (`CoordinationCore.tla`, `AtMostOneHolderPerEpoch`)—the direct analog of a Kafka partition leader, per-key and scoped, never a global lock. A minority partition starves rather than splitting the brain; safety is absolute, availability is sacrificed only in the minority and only for the CP subset.

7 A time sequence of streamdb operations

Figure 1 orders the common and the distinctive streamdb operations in time across a client, two replica/voter nodes, the streamdb substrate (pubsub tip + IPFS segments), and a third node that joins with an empty disk.

❶ A **write** is a signed, content-addressed `put` appended by the ingress node and gossiped as a tip; a peer merges it by set-union and folds it under per-field LWW. ❷ A **concurrent write** to the same field from the peer carries a higher HLC; with no coordination, set-union merge plus highest-HLC-wins converges both nodes to the identical row—the CvRDT property that a broker buys with a single leader, Pillar buys with content-addressing and an HLC. ❸ A **CP exclusive** op (a uniqueness `INSERT` or a hard quota decrement) is gated: the ingress acquires a quorum-fenced grant for that key’s Strict partition before the reducer admits it—a per-key lease, not a global lock. ❹ A **query** is served from the locally folded materialized view via the (query, root) cache; any node that pins the sources answers identically. ❺ A **new view over existing data** folds the unchanged, content-addressed source log from snapshot + tail—no migration, old and new coexist. ❻ A **bootstrap** node with an empty disk rehydrates by fetching the snapshot segment and log tail from *any* peer over IPFS and unsealing with its custody-held key. Modeled against the streamdb and

coordination-core specifications.

8 Placement: the collection is the pinning boundary

Not every cell node must serve every collection; users isolate data to specific nodes for performance, security, or other reasons. Every layer—events, K/V, document, SQL, TSDB—inherits one placement decision because each is a fold of a collection’s event stream, so the **collection** is the pinning unit: pin a collection on a node and that node can serve its document view, its TSDB blocks, and any SQL view whose sources it also holds.

The default (no selector) pins a collection across the *entire cell*: the cell is the replication set, and there is no separate replication-factor knob (it would raise meaningless warnings on small clusters and duplicate what cell membership already expresses). Isolation is opt-in via a `CollectionPlacement` resource that selects nodes by *tag*, reusing the existing attested topology-label machinery that RBAC targets already use; an empty selector means the whole cell. Views follow their sources: a materialized view is servable on a node iff that node pins all of the view’s source collections, so placing a view expands to co-pinning its sources—a node never holds a view without its data. The cell remains the confidentiality boundary; placement only narrows within a cell. The CLI/UI surfaces, per collection, its placement tags and the live count and list of participating nodes.

9 What Pillar has that Samza, Kafka, and Mongo do not

Honestly, including where Pillar loses. **Advantages:** no broker and leaderless (the log is a P2P-gossiped CvRDT, nothing central to run or trust); cryptographically verifiable and tamper-evident (content-addressed Merkle-DAG, per-author hash chains and signatures, so a faulty node is detected); identity, authorization, and encryption in the record format rather than bolted-on ACLs over a trusted operator; a per-stream CAP knob with model-checked convergence and heal-after-partition rather than Kafka’s CP-only partitions; formal verification in TLA⁺; content-addressed durability you self-host (embedded IPFS, off the public DHT), so a consumer bootstraps from any peer with free dedup and no broker to replay from; one envelope and one substrate unifying log, state store, query, and durability instead of five wired-together systems; native subscribe/verify where a client checks a view’s tip itself; and federation/mobility over NAT-traversing libp2p rather than a datacenter-LAN assumption. **Where Pillar loses:** throughput and latency (signing, sealing, content-addressing, and gossiping per op is heavier than a tuned Kafka append,

and Pillar is young and unproven at scale); the conflict model (per-field LWW is weaker than a single-leader total order for some workloads); and maturity (formal proofs are not production hours). Net: Pillar wins on decentralization, verifiability, built-in identity/crypto, formal correctness, and multi-model unification, and does not win on raw throughput or maturity—Samza’s architecture minus the broker, plus cryptographic trust, plus a unified state/query/durability layer.

10 Performance is a standing discipline

The crypto layer (sign + seal + content-address per op) is real per-op overhead and the acknowledged price of the guarantees above; the gap to a plaintext broker is closed by engineering *within* the safety envelope, never by weakening a guarantee. A recurring (daily) self-perpetuating task measures streamdb op throughput/latency and view-fold cost against a tracked baseline on a fixed harness and, on a regression or headroom, files concrete tuning follow-ups—removing redundant hot-path copies/serializations, fusing envelope construction with signing, hardware-accelerated hashing and AEAD (AES-NI/AVX), Ed25519 batch verification, a verify-cache keyed on Cid, zero-copy segment I/O, and batching ops per gossip round. The hard invariant on every tuning task: no change may weaken a guarantee (verifiability, per-op signing, content-addressing, or the per-stream CAP posture); a “speedup” that drops a signature, a hash check, or a seal is a bug, not an optimization.

11 Verification posture

Consistent with Pillar’s specify-then-build rule, the original-design core of this data layer is TLA⁺-gated before it is trusted: the streamdb CvRDT and its fold, the keyed-store op schema including the HLC ordering rule, the coordination core that orders the CP subset, and the confidentiality/seal invariants of the wire format. The SQL/query compilation, the catalog documents, the placement resource, and CLI/UI ergonomics ship test-driven against a live node—including a real remote read/query against a running node, not a fixed-kind HTTP shim—so the offer-every-primitive-as-an-inspectable-service doctrine is exercised by a gate, not merely asserted.

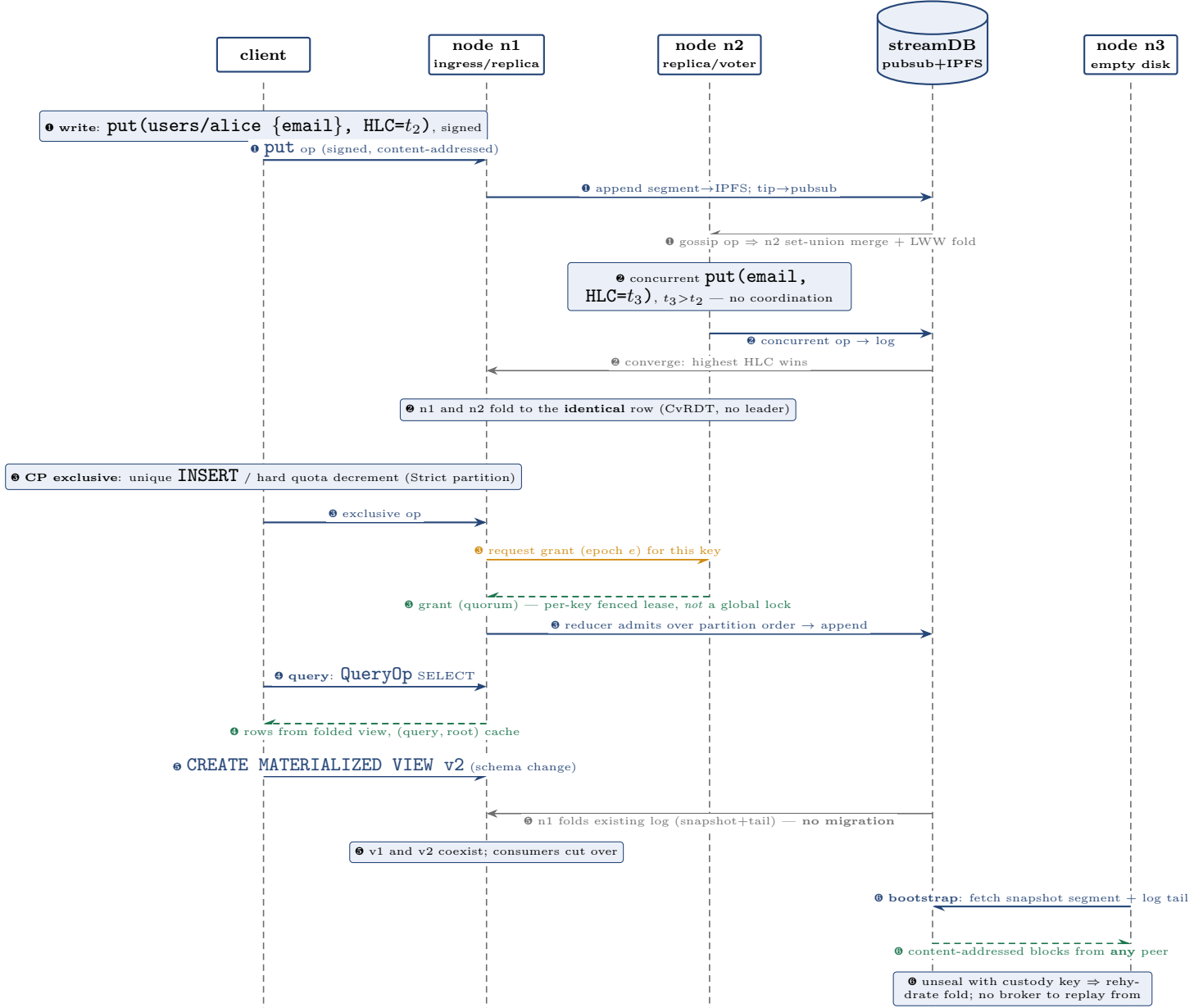


Figure 1: Time sequence of streamdb operations, time flowing downward. ❶ write + gossip merge; ❷ concurrent same-field write converging by highest-HLC last-writer-wins with no coordination (the distinctive CvRDT property); ❸ a CP exclusive admission gated on a per-key quorum-fenced lease from the coordination core, not a global lock; ❹ a query served from a folded, root-cached materialized view; ❺ a new view built over the unchanged content-addressed source log with no migration; ❻ a fresh node rehydrating from IPFS-pinned snapshot + tail fetched from any peer and unsealed with its custody key. Common operations (❶,❹) and Pillar-unique ones (❷,❸,❺,❻) ride the one PillarMessage substrate.